PYTHON · OOP · STUDY GUIDE

PYTHON FUNDAMENTALS FOR OOP

9 topics · code and output · clear explanations

Name	**BOEUN Piseth**
ID	**e20241245**
Group	**I3-AMS-1A**
Course	**Object-Oriented Programming (OOP)**
Lecturer	**VANN Visal**

Contents

This guide turns the original slides into a study document. Each topic starts with why it matters in OOP, then walks through short sections with key points, code, output and an explanation.

#	Topic	What you will learn
1	**Python Basic Data Types**	int, float, bool, str, list, tuple, dict, set, references, literals
2	**Python Variables and Attributes**	Local variables, instance attributes, class attributes
3	**Modifiers and Access Control, Part 1**	Public / non-public / private names, static and class methods
4	**Modifiers and OOP Features, Part 2**	Abstract classes, encapsulation, @property, @final
5	**Python Operators, Part 1**	Arithmetic, comparison, bitwise and logical operators
6	**Python Operators, Part 2**	Assignment, identity, membership, precedence
7	**Python Decision-Making Statements**	if, if-else, if-elif-else
8	**Python Decision-Making, Part 2**	Nested if, match-case, conditional expressions, menu system
9	**Python Loops and Loop Control**	while, for, break, continue, pass

> **How to read the code boxes:** The teal CODE box is what you type. The dark OUTPUT box is exactly what Python prints. Lines starting with `#` are comments and are ignored by Python.

TOPIC 1 OF 9

Python Basic Data Types

> **Why it matters in OOP:** Every attribute you store in an object is a value of some type. Knowing whether a type can change, and whether it is copied or shared, explains many surprising bugs later.

Primitive Types

- `int`: whole numbers of any size, such as 20 or -3.
- `float`: decimal numbers such as 85.5, used for scores and averages.
- `bool`: only `True` or `False`, used in conditions.
- `str`: text inside quotes. Strings are immutable, so a change creates a new string.

CODE

```
age = 20          # integer
score = 85.5      # float
passed = True     # boolean
name = "Panha"    # string
print(type(age), type(score))
print(type(passed), type(name))
```

OUTPUT

```
<class 'int'> <class 'float'>
<class 'bool'> <class 'str'>
```

How it works. Python has four basic types. The built-in `type()` function tells you the type of any value, and the output confirms what each variable holds. Python decides the type from the value you assign, so you never declare it yourself.

> **Watch out:** Immutable means the original string never changes. A call such as `name.upper()` returns a new string; it does not modify `name`.

Collections: list and tuple

- `list` `[ ]` keeps items in order and can be changed.
- `tuple` `( )` keeps items in order but cannot be changed after creation.
- Both use an index that starts at 0.
- Use a list for scores that grow; use a tuple for fixed data such as (name, age).

CODE

```
scores = [85.5, 90, 72]   # list
scores.append(88)         # add a new item
info = ("Panha", 20)      # tuple
print(scores[0], len(scores))
print(info[1])
# info[1] = 21   -> TypeError
```

OUTPUT

```
85.5 4
20
```

How it works. After `append(88)` the list holds four items, so `len(scores)` is 4 and `scores[0]` is the first item. The tuple cannot be modified, so trying `info[1] = 21` raises a TypeError. That protection is the reason to prefer tuples for data that must not change.

Collections: dict, set and objects

- `dict` stores key-value pairs; you look up a value by its key.
- `set` stores unique items only; duplicates are removed.
- An object is an instance of a class; a variable holds a reference to it.
- Two variables can refer to the same object, so a change shows through both.

CODE

```
student = {"name": "Dara", "age": 20}   # dict
print(student["name"])
ids = {101, 102, 101}                   # set
print(ids)
a = [1, 2]
b = a
b.append(3)
print(a)
```

OUTPUT

```
Dara
{101, 102}
[1, 2, 3]
```

How it works. A dict finds a value by its key, and a set silently drops the duplicate 101. The last example is the important one: `b = a` does not copy the list, it makes `b` point to the same list. So `b.append(3)` also changes what `a` sees.

> **Watch out:** To get a real, independent copy of a list, use `b = a.copy()` or `b = a[:]`.

Literals

- A literal is a value written directly in the code.
- Numeric: `10`, `3.14`, `1e3`. String: `'A'`, `"text"`, `"""multi-line"""`.
- Boolean: `True` and `False`. Special: `None` means 'no value'.
- Collections: `[1, 2]`, `(1, 2)`, `{"a": 1}`, `{1, 2}`.

CODE

```
x = 10            # int literal
y = 3.14          # float literal
s = 'Panha'       # string literal
ok = True         # Boolean literal
nothing = None    # None literal
items = [1, 2]    # list literal
d = {"a": 1}      # dict literal
```

OUTPUT

```
(no output: this code
only creates values)
```

How it works. Each line writes a value directly, which is what makes it a literal. Collections have their own literal shapes: square brackets for a list, round brackets for a tuple, and curly braces for a dict or a set.

★ Practical Example: Student Records

- Name is a `str`, age an `int`, score a `float`.
- Each student record is a `dict` holding those values.
- All records are stored together in one `list`.
- A `for` loop reads each record in turn.

CODE

```
students = [
    {"name": "Dara", "age": 20, "score": 85.5},
    {"name": "Sok",  "age": 21, "score": 72.0},
]
for s in students:
    print(s["name"], s["score"])
```

OUTPUT

```
Dara 85.5
Sok 72.0
```

How it works. This example combines the whole topic. Each student is a dict (str name, int age, float score), and all the dicts live in one list. The loop visits each record and prints its name and score. This list-of-dicts pattern is the same idea a class will formalise later.

> **Key takeaway:** Choose the type that matches the data: list for growing data, tuple for fixed data, dict for labelled data, set for unique data. Remember that variables hold references, not copies.

TOPIC 2 OF 9

Python Variables and Attributes

> **Why it matters in OOP:** Attributes are how an object remembers things. Knowing where a variable lives (inside a method, on one object, or on the whole class) tells you who can see it and how long it lasts.

Local Variables

- Created inside a method or function and used only there.
- Disappears when the method finishes.
- Not stored with `self`, so other methods cannot see it.
- Use it for temporary calculations.

CODE

```
class Student:
    def average(self, a, b):
        total = a + b      # local
        return total / 2

s = Student()
print(s.average(80, 90))
# print(total)  -> NameError
```

OUTPUT

```
85.0
```

How it works. `total` exists only while `average()` is running. Once the method returns, the variable is gone, so printing it outside raises a NameError. The result 85.0 is a float because `/` is true division.

Instance Attributes

- Created with `self.name = value`, usually inside `__init__`.
- Every object keeps its own copy.
- Changing one object does not affect the others.
- Read with `object.name`.

CODE

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

a = Student("Panha", 20)
b = Student("Seth", 21)
print(a.name, a.age)
print(b.name, b.age)
```

OUTPUT

```
Panha 20
Seth 21
```

How it works. `__init__` runs automatically when an object is created and sets up its data. Object `a` stores Panha and 20, while object `b` stores Seth and 21. Each object has its own separate copy, so changing one never affects the other.

Class Attributes

- Defined in the class body, outside any method.
- One copy is shared by all instances.
- Read with `Student.school` or `obj.school`.
- `Student.school = ...` updates it for every student.

CODE

```
class Student:
    school = "Royal School"

a = Student()
b = Student()
print(a.school, b.school)
Student.school = "ABC School"
print(a.school, b.school)
```

OUTPUT

```
Royal School Royal School
ABC School ABC School
```

How it works. A class attribute is written once in the class body and shared by every object. Both `a` and `b` first show Royal School. Changing it through the class, `Student.school = 'ABC School'`, updates the value for every student at once.

Instance vs Class Attribute

- `a.school = 'X'` creates a new instance attribute only on `a`.
- The class attribute and the other objects stay unchanged.
- Python looks in the object first, then in the class.
- Use class attributes for shared data, instance attributes for personal data.

CODE

```
class Student:
    school = "Royal School"

a = Student()
b = Student()
a.school = "Private School"
print(a.school, b.school)
```

OUTPUT

```
Private School Royal School
```

How it works. Writing `a.school = 'Private School'` does not edit the class attribute. It creates a brand-new attribute that lives only on `a`, and that new one hides the class value for `a`. Object `b` has no such attribute, so Python falls back to the class and finds Royal School.

> **Watch out:** This is a very common source of confusion. To change the shared value for everyone, assign through the class name, not through an object.

★ Practical Example: Student Class

- `school` is a class attribute shared by every student.
- `name` is an instance attribute, different for each student.
- `total` is a class attribute that counts how many students exist.
- Each new object adds 1 to `Student.total`.

CODE

```
class Student:
    school = "ITC"
    total = 0

    def __init__(self, name):
        self.name = name
        Student.total += 1

a = Student("Panha")
b = Student("Sok")
print(a.name, b.name, Student.school, Student.total)
```

OUTPUT

```
Panha Sok ITC 2
```

How it works. All three kinds of data appear here. `school` is shared, `name` is personal to each object, and `total` is a shared counter. Because `__init__` adds 1 to `Student.total` each time an object is built, the counter reads 2 after two students are created.

> **Key takeaway:** Local variables are temporary, instance attributes belong to one object, and class attributes are shared by all objects. Assigning through an object never changes the class attribute; it creates a new instance attribute.

TOPIC 3 OF 9

Modifiers and Access Control, Part 1

> **Why it matters in OOP:** Good classes protect their data. Python does this mostly through naming conventions, and it gives you static and class methods to organise behaviour that does not belong to one object.

Public, Non-Public and Private Names

- Python has no real `private` keyword; it uses naming rules.
- `name`: public, free to use anywhere.
- `_name`: non-public, a hint that it is for internal use.
- `__name`: private, Python renames it (name mangling).

CODE

```
class Account:
    def __init__(self):
        self.owner = "Panha HIN"
        self._branch = "PP"
        self.__pin = 261206

acc = Account()
print(acc.owner, acc._branch)
```

OUTPUT

```
Panha HIN PP
```

How it works. `owner` is public. `_branch` has one underscore, which tells other programmers 'please do not touch this from outside', but Python still lets you read it. `__pin` has two underscores and is treated as private through name mangling, which the next section explains.

Access Control and Name Mangling

- Inside the class, `__pin` is renamed to `_Account__pin`.
- Writing `a.__pin` from outside raises `AttributeError`.
- It is still reachable as `a._Account__pin`: Python trusts the programmer.
- So access control is by convention; use methods to read or change private data.

CODE

```
class Account:
    def __init__(self):
        self.__pin = 1234

a = Account()
print(a._Account__pin)
print(a.__pin)
```

OUTPUT

```
1234
AttributeError: 'Account' object
has no attribute '__pin'
```

How it works. Python quietly renames `__pin` to `_Account__pin` inside the class, so the plain name `a.__pin` is not found and an AttributeError is raised. The renamed version still works, which proves that Python's 'private' is about avoiding accidents, not about security.

> **Watch out:** Never rely on `__` to hide secrets. It only prevents accidental access and name clashes in subclasses.

The @staticmethod Decorator

- Declared with `@staticmethod`; it takes no `self` or `cls`.
- It cannot read object or class data.
- It groups a related utility function inside the class.
- Call it with `ClassName.method()`.

CODE

```
class MathTool:
    @staticmethod
    def is_even(n):
        return n % 2 == 0

print(MathTool.is_even(4))
print(MathTool.is_even(7))
```

OUTPUT

```
True
False
```

How it works. A static method behaves like an ordinary function that happens to live inside the class for organisation. It only works with the values you pass in. `is_even(4)` is True because 4 % 2 is 0, and `is_even(7)` is False because 7 % 2 is 1.

The @classmethod Decorator

- Declared with `@classmethod`; the first parameter is `cls`, the class itself.
- It can read and change class attributes.
- Often used as an alternative constructor.
- It also works when called from a subclass.

CODE

```
class Student:
    school = "Royal School"

    def __init__(self, name):
        self.name = name

    @classmethod
    def from_text(cls, text):
        return cls(text.title())

    @classmethod
    def get_school(cls):
        return cls.school

s = Student.from_text("dara")
print(s.name, Student.get_school())
```

OUTPUT

```
Dara Royal School
```

How it works. A class method receives `cls`, so it can create objects or read shared data. `from_text('dara')` cleans the text with `title()` and builds a Student named Dara, which is an alternative way to construct an object. `get_school()` simply returns the shared school name.

> **Watch out:** Use `cls(...)` rather than hard-coding `Student(...)`, so the method still builds the right type when a subclass calls it.

★ Practical Example: BankAccount

- The balance is private (`__balance`) and changes only through methods.
- `deposit` checks the amount with the static method `is_valid`.
- The class method `bank_name` returns shared class data.

CODE

```
class BankAcc:
    bank = "ABA"

    def __init__(self, owner, balance):
        self.owner = owner
        self.__balance = balance

    @staticmethod
    def is_valid(amount):
        return amount > 0

    def deposit(self, amount):
        if self.is_valid(amount):
            self.__balance += amount

    def get_balance(self):
        return self.__balance

    @classmethod
    def bank_name(cls):
        return cls.bank

acc = BankAcc("Panha", 100)
acc.deposit(82)
print(acc.get_balance(), BankAcc.bank_name(),
acc.owner)
```

OUTPUT

```
182 ABA Panha
```

How it works. The balance is private, so it changes only through `deposit()`, which uses the static method `is_valid` to accept positive amounts only. `get_balance()` reads the private data and `bank_name()` returns the shared class data. Starting from 100 and depositing 82 gives 182.

> **Key takeaway:** Underscores are conventions, not locks. Keep sensitive data private and expose it through methods. Use a static method for a helper that needs no data, and a class method when you need the class itself.

TOPIC 4 OF 9

Modifiers and OOP Features, Part 2

> **Why it matters in OOP:** These features let you design classes that other programmers cannot misuse: templates that force a structure, controlled access to data, and clean attribute-style syntax with validation.

Abstract Classes and Methods

- An abstract class is a template; you cannot create objects from it directly.
- Inherit from `ABC` (module `abc`) to make one.
- `@abstractmethod` marks a method that every subclass must override.
- A subclass that skips it cannot be instantiated.

CODE

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

s = Shape()
```

OUTPUT

```
TypeError: Can't instantiate
abstract class Shape ...
```

How it works. `Shape` defines what every shape must be able to do, but not how. Because `area()` is abstract, Python refuses to build a `Shape` object, and any subclass that forgets to implement `area()` cannot be built either. The exact wording of the error depends on your Python version.

Access Control and Encapsulation

- Encapsulation puts data and the methods that use it in one class.
- Hide internal data with `_x` or `__x`.
- Expose controlled access through methods.
- This protects the data from invalid values.

CODE

```
class Student:
    def __init__(self, score):
        self.__score = score

    def set_score(self, value):
        if 0 <= value <= 100:
            self.__score = value

    def get_score(self):
        return self.__score

s = Student(70)
s.set_score(150)
print(s.get_score())
```

OUTPUT

```
70
```

How it works. The score is hidden behind `__score` and can change only through `set_score()`, which accepts values from 0 to 100. The call `set_score(150)` is rejected, so the score stays at 70. The class protects its own data instead of trusting every caller.

The @property Decorator

- `@property` turns a method into an attribute you read without parentheses.
- Add `@score.setter` to allow updates with validation.
- Same safety as getters and setters, with cleaner syntax.
- Callers write `s.score`, not `s.get_score()`.

CODE

```
class Student:
    def __init__(self, score):
        self._score = score

    @property
    def score(self):
        return self._score

    @score.setter
    def score(self, v):
        if 0 <= v <= 100:
            self._score = v

s = Student(70)
s.score = 90
print(s.score)
```

OUTPUT

```
90
```

How it works. From the outside, `s.score` looks like a normal attribute, but reading and writing actually run the getter and setter methods. The setter checks the range before saving, so `s.score = 90` is accepted. An invalid value such as 150 would be ignored, exactly like `set_score()` earlier.

> **Watch out:** Many programmers prefer `@property` to separate get/set methods because the class can add validation later without changing how other code uses it.

@final and typing.final

- `typing.final` marks a method or class that must not be overridden or subclassed.
- It is a hint for type checkers such as `mypy`.
- Python does not enforce it at runtime.
- In Java, `final` is enforced by the compiler.

CODE

```
from typing import final

class Base:
    @final
    def show(self):
        print("Fixed")

class Child(Base):
    def show(self):      # mypy reports an error
        print("Changed")

Child().show()
```

OUTPUT

```
Changed
```

How it works. `@final` states the programmer's intention, and tools like mypy will warn you. Python itself still runs the code, so `Child` overrides `show()` and prints Changed. This is a key difference from Java, where the compiler would stop you.

★ Practical Example: Shape

- `Shape` sets the rule: every shape must have `area()`.
- `Circle` and `Rectangle` each implement it differently.
- The loop calls `area()` without caring about the type (polymorphism).
- `Shape()` itself can never be created.

CODE

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, r):
        self.r = r
    def area(self):
        return 3.14 * self.r ** 2

class Rectangle(Shape):
    def __init__(self, w, h):
        self.w, self.h = w, h
    def area(self):
        return self.w * self.h

for s in (Circle(2), Rectangle(3, 4)):
    print(s.area())
```

OUTPUT

```
12.56
12
```

How it works. `Shape` is abstract and demands an `area()` method. The circle computes 3.14 x 2 x 2 = 12.56 and the rectangle computes 3 x 4 = 12. The loop treats both the same way and just calls `area()`; each object responds in its own way, which is polymorphism.

> **Key takeaway:** Abstract classes force a contract, encapsulation protects data, `@property` gives clean validated attributes, and `@final` is only a hint to type checkers in Python.

TOPIC 5 OF 9

Python Operators, Part 1

Why it matters in OOP: Operators are how objects and values are combined, compared and tested. Almost every method you write will use them to calculate results or make decisions.

Arithmetic Operators

- `+`, `-`, `*` add, subtract and multiply.
- `/` is true division and always gives a float; `//` drops the decimals.
- `%` gives the remainder; `**` is power.
- Mixing `int` and `float` gives a float.

CODE

```
print(7 + 2, 7 - 2, 7 * 2)
print(7 / 2, 7 // 2)
print(7 % 2, 7 ** 2)
```

OUTPUT

```
9 5 14
3.5 3
1 49
```

How it works. `+`, `-` and `*` work as usual. `7 / 2` is 3.5 but `7 // 2` is 3 because the decimals are dropped. `7 % 2` is the remainder 1, and `7 ** 2` is 7 squared, which is 49.

Comparison Operators

- A comparison compares two values; the result is a `bool`.
- `==` equal, `!=` not equal.
- `>`, `<`, `>=`, `<=` compare order.
- Chaining is allowed: `0 <= score <= 100`.

CODE

```
score = 78
print(score == 78, score != 78)
print(score > 50, score <= 50)
print(0 <= score <= 100)
```

OUTPUT

```
True False
True False
True
```

How it works. Every comparison produces True or False. Python also allows chained comparisons, so `0 <= score <= 100` reads just like mathematics and checks that the score is inside the range.

Watch out: Do not confuse `==` (compare) with `=` (assign).

Bitwise Operators

- They work on the binary bits of integers.
- `&` AND, `|` OR, `^` XOR, `~` NOT.
- `<<` shifts bits left (times 2), `>>` shifts right (divide by 2).
- Here 5 is `101` and 3 is `011` in binary.

CODE

```
print(5 & 3)
print(5 | 3)
print(5 ^ 3)
print(~5)
print(5 << 1, 5 >> 1)
```

OUTPUT

```
1
7
6
-6
10 2
```

How it works. Line up the bits: 101 and 011. AND keeps a 1 only where both have 1 (001 = 1). OR keeps a 1 where either has one (111 = 7). XOR keeps a 1 where they differ (110 = 6). `~5` flips all bits and gives -6. Shifting left by one doubles 5 to 10, and shifting right halves it to 2.

Logical Operators

- `and`: True only if both conditions are True.
- `or`: True if at least one condition is True.
- `not`: reverses the value.
- Short-circuit: Python stops as soon as the result is known.

CODE

```
age, score = 20, 78
print(age >= 18 and score >= 50)
print(age < 18 or score >= 50)
print(not (score >= 50))
```

OUTPUT

```
True
True
False
```

How it works. With age 20 and score 78, the first test has two True parts, so `and` is True. In the second, `age < 18` is False but `score >= 50` is True, so `or` is True. `not` flips True into False. Because of short-circuiting, Python skips the second half of an `and` when the first half is already False.

★ Practical Example: Grades and Eligibility

- The average uses `+` and `/`.
- A comparison checks the passing mark.
- `and` combines the score rule and the attendance rule.
- The result is stored as a `bool`.

CODE

```
math_score, python_score = 80, 70
average = (math_score + python_score) / 2
attendance = 90
eligible = average >= 50 and attendance >= 75
print(average, eligible)
```

OUTPUT

```
75.0 True
```

How it works. The average is (80 + 70) / 2 = 75.0. A student is eligible only when the average passes 50 and attendance reaches 75. Both rules are met, so `eligible` is True. The variables are named `math_score` and `python_score` so they do not clash with Python's own `math` module name.

> **Key takeaway:** Use `/` for a float and `//` for whole division, remember that comparisons return a bool, and combine conditions with `and`, `or`, `not`. Bitwise operators work on binary digits, not on whole values.

TOPIC 6 OF 9

Python Operators, Part 2

> **Why it matters in OOP:** These operators update object state, compare identity, test membership, and choose values in one line. They appear constantly inside methods that change attributes.

Assignment Operators

- `=` stores a value in a variable.
- Compound operators update in place: `x += 5` means `x = x + 5`.
- Others: `-=`, `*=`, `/=`, `//=`, `%=`, `**=`.
- They also work on attributes, such as `self.score += 5`.

CODE

```
x = 10
x += 5
x *= 2
x -= 3
print(x)
x //= 4
print(x)
```

OUTPUT

```
27
6
```

How it works. Trace the value step by step: 10 becomes 15, then 30, then 27, which is printed. Then `x //= 4` divides 27 by 4 and drops the decimals, leaving 6.

Identity and Membership Operators

- `is` checks if two names point to the same object; `==` checks equal values.
- `is not` is the opposite of `is`.
- `in` checks whether an item is inside a list, string, set or dict keys.
- `not in` is the opposite of `in`.

CODE

```
a = [1, 2]
b = [1, 2]
c = a
print(a == b, a is b, a is c)
print(2 in a, 5 not in a)
print("P" in "Panha")
```

OUTPUT

```
True False True
True True
True
```

How it works. `a` and `b` hold equal values, so `a == b` is True, but they are two separate lists, so `a is b` is False. `c = a` points to the same list, so `a is c` is True. `in` and `not in` simply ask whether something is present.

> **Watch out:** Use `is` mainly for `None` checks (`x is None`). For comparing values, use `==`.

Conditional Expressions

- Syntax: `value_if_true if condition else value_if_false`.
- Shorter than a full if-else for a simple choice.
- Java equivalent: `condition ? a : b`.
- Keep it simple; avoid long nested ones.

CODE

```
score = 72
status = "Pass" if score >= 50 else "Fail"
print(status)
grade = "A" if score >= 80 else "B"
print(grade)
```

OUTPUT

```
Pass
B
```

How it works. The expression picks one of two values in a single line. With a score of 72 the first test is True, so `status` is Pass. The second test `72 >= 80` is False, so `grade` becomes B.

Operator Precedence and Associativity

- Precedence decides which operator runs first.
- High to low: `**`, `* / // %`, `+ -`, comparisons, `not`, `and`, `or`.
- Associativity: equal operators go left to right; `**` goes right to left.
- Use parentheses to make your intent clear.

CODE

```
print(2 + 3 * 4)
print((2 + 3) * 4)
print(2 ** 3 ** 2)
print(10 - 4 - 3)
print(not True or True)
```

OUTPUT

```
14
20
512
3
True
```

How it works. Multiplication runs before addition, so `2 + 3 * 4` is 14, while parentheses force `(2 + 3) * 4` to be 20. Subtraction goes left to right: (10 - 4) - 3 = 3. Power goes right to left: `2 ** 3 ** 2` is 2 ** 9 = 512. In the last line, `not` runs first, giving False or True, which is True.

★ Practical Example: Updating and Checking Data

- `+=` updates an object attribute.
- `in` checks membership in a list.
- `and` joins two conditions.
- A conditional expression picks the label.

CODE

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score

subjects = ["Math", "Python"]
s = Student("Panha", 70)
s.score += 10
status = "Excellent" if "Python" in subjects and
s.score >= 50 else "Keep trying"
print(s.name, s.score, status)
```

OUTPUT

```
Panha 80 Excellent
```

How it works. This example uses the whole group. `+=` raises the score from 70 to 80, `in` checks the subjects list, `and` joins two conditions, and a conditional expression picks the label. Both tests are True, so the status is Excellent.

> **Key takeaway:** Use `==` for equal values and `is` for the same object. Compound assignment saves typing, `in` tests membership, and parentheses make precedence clear.

TOPIC 7 OF 9

Python Decision-Making Statements

> **Why it matters in OOP:** Objects must react to different situations, such as rejecting a withdrawal or assigning a grade. Decision statements let your methods choose what to do.

The if Statement

- Runs the indented block only if the condition is True.
- The `if` line ends with a colon `:`.
- Indentation (4 spaces) marks the block.
- If the condition is False, Python skips the block.

CODE

```
score = 72
if score >= 50:
    print("You passed")
print("Done")
```

OUTPUT

```
You passed
Done
```

How it works. The indented line belongs to the `if`, so it runs only when the condition is True. With a score of 72 it prints You passed. `Done` is not indented, so it always runs.

The if-else Statement

- `else` handles the case when the condition is False.
- Exactly one of the two blocks runs.
- `else` has no condition of its own.

CODE

```
score = 40
if score >= 50:
    print("Pass")
else:
    print("Fail")
```

OUTPUT

```
Fail
```

How it works. Python tests the condition once and then runs exactly one block. With a score of 40, `40 >= 50` is False, so the `else` block runs and prints Fail.

The if-elif-else Statement

- Use it when there are more than two outcomes.
- Python checks from top to bottom and runs the first True block.
- `elif` can be repeated; `else` is the optional default.
- Order matters: test the highest range first.

CODE

```
score = 72
if score >= 80:
    grade = "A"
elif score >= 70:
    grade = "B"
elif score >= 50:
    grade = "C"
else:
    grade = "F"
print(grade)
```

OUTPUT

```
B
```

How it works. The first test `72 >= 80` fails, the second `72 >= 70` passes, so grade is B and Python skips the rest of the chain.

> **Watch out:** If you test `score >= 50` first, every score above 50 would match it and the higher grades would never be reached.

★ Practical Example: Pass/Fail and Letter Grades

- `if-else` decides pass or fail.
- `if-elif-else` assigns the letter grade.
- The loop repeats both decisions for three scores.

CODE

```
for score in (85, 72, 45):
    if score >= 50:
        status = "Pass"
    else:
        status = "Fail"

    if score >= 80:
        grade = "A"
    elif score >= 70:
        grade = "B"
    else:
        grade = "F"
    print(score, status, grade)
```

OUTPUT

```
85 Pass A
72 Pass B
45 Fail F
```

How it works. The loop runs two separate decisions for each score. The first chooses Pass or Fail using 50 as the passing mark, and the second picks the letter grade. This gives 85 Pass A, 72 Pass B and 45 Fail F.

Key takeaway: `if` runs a block when a condition is True, `else` handles the rest, and `elif` handles several ranges. Python checks from the top, so order your tests carefully, and always indent consistently.

TOPIC 8 OF 9

Python Decision-Making, Part 2

> **Why it matters in OOP:** Real programs need dependent checks and menus. Nested conditions, `match-case` and compact conditional expressions keep that logic readable inside your classes.

Nested if Statements

- A nested `if` is an `if` placed inside another `if`.
- The inner condition is checked only when the outer one is True.
- Good for dependent checks, such as age first, then score.
- Very deep nesting is hard to read; use `and` when possible.

CODE

```
age, score = 20, 72
if age >= 18:
    if score >= 50:
        print("Eligible")
    else:
        print("Low score")
else:
    print("Too young")
```

OUTPUT

```
Eligible
```

How it works. Age is checked first. Because 20 is at least 18, Python goes inside and checks the score. 72 passes, so the output is Eligible. If the age test had failed, the score would never have been looked at.

Multiple Branches with match-case

- `match` compares one value against several `case` patterns (Python 3.10+).
- The first matching case runs; no `break` is needed.
- `case _` is the default branch.
- It replaces long `if-elif-else` chains.

CODE

```
day = 3
match day:
    case 1:
        print("Monday")
    case 2:
        print("Tuesday")
    case 3:
        print("Wednesday")
    case _:
        print("Other")
```

OUTPUT

```
Wednesday
```

How it works. `match` looks at the value of `day` and runs the first case that fits. Since `day` is 3, it prints Wednesday. The underscore case catches everything that did not match, like `else`.

> **Watch out:** `match-case` needs Python 3.10 or newer. On older versions, use if-elif-else.

More match-case Patterns

- Combine values with `|`: `case 'add' | 'new':`.
- Add a guard condition: `case n if n > 90:`.
- Match sequences: `case (x, y):`.
- Before Python 3.10, use `if-elif-else` instead.

CODE

```
cmd = "exit"
match cmd:
    case "add" | "new":
        print("Adding")
    case "exit" | "quit":
        print("Bye")
    case _:
        print("Unknown")
```

OUTPUT

```
Bye
```

How it works. The `|` symbol means 'or', so one case can accept several values. Here `cmd` is 'exit', which matches the second case, so the output is Bye. Guards and sequence patterns make `match` much more powerful than a simple switch.

Conditional Expression: Java vs Python

- Java: `condition ? value1 : value2`.
- Python: `value1 if condition else value2`.
- The condition sits in the middle in Python.
- It returns a value, so you can assign or print it directly.

CODE

```
age = 20
mess = "Adult" if age >= 18 else "Minor"
print(mess)
print("Even" if age % 2 == 0 else "Odd")
```

OUTPUT

```
Adult
Even
```

How it works. Both languages do the same job but write it in a different order. Because the expression returns a value, you can store it in a variable or place it directly inside `print()`. Age 20 is at least 18 and divisible by 2, so the results are Adult and Even.

★ Practical Example: Student Menu System

- `while True` keeps the menu running.
- `match` selects the action.
- A nested `if` checks that the name is not empty.
- `break` leaves the loop on option 3.

CODE

```
students = []
while True:
    print("Choose a choice: 1.Add 2.Show 3.Exit")
    choice = input("Choose: ")
    match choice:
        case "1":
            name = input("Name: ")
            if name != "":
                students.append(name)
        case "2":
            print(students)
        case "3":
            break
        case _:
            print("Invalid")
print("Exit program")
```

OUTPUT

```
Choose a choice: 1.Add 2.Show
3.Exit
Choose: 1
Name: Panha
Choose a choice: 1.Add 2.Show
3.Exit
Choose: 2
['Panha']
Choose a choice: 1.Add 2.Show
3.Exit
Choose: 3
Exit program
```

How it works. `while True` keeps showing the menu until the user chooses 3. Option 1 uses a nested `if` to reject an empty name, option 2 shows the list, and option 3 uses `break` to leave the loop. `case _` handles any invalid input.

> **Watch out:** `input()` always returns a string, so the cases are written as `'1'`, `'2'`, `'3'`. If you convert with `int()`, the cases must be numbers instead, otherwise nothing will match.

> **Key takeaway:** Use nested `if` for dependent checks but flatten with `and` when you can. `match-case` replaces long chains for one value. Python's conditional expression puts the condition in the middle.

TOPIC 9 OF 9

Python Loops and Loop Control

> **Why it matters in OOP:** Objects often hold collections, and loops let you process every item: totals, searches and menus. Loop control then lets you skip, stop or leave a placeholder.

The while Loop

- Repeats as long as the condition is True.
- The condition is checked before every round.
- Change the variable inside the loop, or it never ends.
- Best when the number of rounds is unknown.

CODE

```
count = 1
while count <= 3:
    print("Round", count)
    count += 1
```

OUTPUT

```
Round 1
Round 2
Round 3
```

How it works. Each round checks `count <= 3` first. The line `count += 1` moves the loop toward its end; after round 3, `count` becomes 4, the condition is False and the loop stops.

> **Watch out:** If you forget `count += 1`, the condition stays True forever and you get an infinite loop.

The for Loop

- Goes through each item of a sequence: a list, string or `range()`.
- There is no counter to manage by hand.
- `range(1, 4)` gives 1, 2, 3; the end value is excluded.
- `enumerate()` gives the index and the item together.

CODE

```
for n in range(1, 4):
    print(n)
for ch in "Py":
    print(ch)
names = ["Dara", "Sok"]
for i, name in enumerate(names):
    print(i, name)
```

OUTPUT

```
1
2
3
P
y
0 Dara
1 Sok
```

How it works. A `for` loop takes the items one by one from any sequence. `range(1, 4)` stops before 4, a string gives one character at a time, and `enumerate()` is handy when you need both the position and the value.

Repetition: for vs while

- `for`: use it when you know the items or the number of rounds.
- `while`: use it when you repeat until a condition changes.
- Python has no do-while loop.
- Simulate it with `while True` and `break`, so the body runs at least once.

CODE

```
while True:
    n = int(input("Enter n > 0: "))
    if n > 0:
        break
print("Accepted", n)
```

OUTPUT

```
Enter n > 0: -2
Enter n > 0: 5
Accepted 5
```

How it works. The body always runs once before the first test, which is exactly what a do-while loop does. The input -2 fails the check, so the loop asks again. The input 5 passes, `break` ends the loop, and the program prints Accepted 5.

Loop Control: break, continue and pass

- `break` stops the whole loop immediately.
- `continue` skips the rest of this round and starts the next.
- `pass` does nothing; it is a placeholder for empty code.
- `break` and `continue` usually sit inside an `if`.

CODE

```
for n in range(1, 8):
    if n == 2:
        continue
    if n == 5:
        break
    print(n)
```

OUTPUT

```
1
3
4
```

How it works. When n is 2, `continue` skips the print and moves on to 3. When n reaches 5, `break` ends the whole loop, so 5, 6 and 7 are never printed. The output is 1, 3, 4.

★ Practical Example: Averages and Searching

- A `for` loop visits each student object.
- A running total gives the average.
- A second loop searches by name.
- `break` stops the search once the student is found.

CODE

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score

students = [Student("Panha", 80), Student("Sok", 65)]

total = 0
for s in students:
    total += s.score
print(total / len(students))

for n in students:
    if n.name == "Panha":
        print("Found", n.name, n.score)
        break
```

OUTPUT

```
72.5
Found Panha 80
```

How it works. The first loop adds every score to a running total: 80 + 65 = 145, and 145 / 2 gives the average 72.5. The second loop searches by name. It finds Panha on the first item, prints Found Panha 80, and `break` stops the search so no more students are checked.

> **Key takeaway:** Use `for` when you know the items, `while` when you repeat until something changes. Python has no do-while, so use `while True` with `break`. `break` stops, `continue` skips one round, and `pass` does nothing.

References

- Python Tutorial: https://docs.python.org/3/tutorial/
- Loops and control flow: https://docs.python.org/3/tutorial/controlflow.html
- Conditions: https://www.learnpython.org/en/Conditions
- Basic operators: https://www.learnpython.org/en/Basic_Operators
- Classes and objects: https://www.learnpython.org/en/Classes_and_Objects

Thank you